Studio Nx

P1-28 비밀번호 찾기 API — 함수 설명 합본 (백엔드 150~164)

2026-10-15

목차

1. ai-notes/backend(fastAPI_Python)/P1-28 - 비밀번호 찾기/150_security_hash_reset_token.md
2. ai-notes/backend(fastAPI_Python)/P1-28 - 비밀번호 찾기/151_security_generate_reset_token.md
3. ai-notes/backend(fastAPI_Python)/P1-28 - 비밀번호 찾기/152_password_reset_service_create_reset_token.md
4. ai-notes/backend(fastAPI_Python)/P1-28 - 비밀번호 찾기/153_password_reset_service_check_reset_rate_limit.md
5. ai-notes/backend(fastAPI_Python)/P1-28 - 비밀번호 찾기/154_password_reset_service_request_password_reset.md
6. ai-notes/backend(fastAPI_Python)/P1-28 - 비밀번호 찾기/155_password_reset_service_reset_password.md
7. ai-notes/backend(fastAPI_Python)/P1-28 - 비밀번호 찾기/156_security_issued_before_password_change.md
8. ai-notes/backend(fastAPI_Python)/P1-28 - 비밀번호 찾기/157_security_decode_token_with_iat.md
9. ai-notes/backend(fastAPI_Python)/P1-28 - 비밀번호 찾기/158_deps_get_current_user_password_reset.md
10. ai-notes/backend(fastAPI_Python)/P1-28 - 비밀번호 찾기/159_auth_refresh_password_reset.md
11. ai-notes/backend(fastAPI_Python)/P1-28 - 비밀번호 찾기/160_auth_password_reset_request_route.md
12. ai-notes/backend(fastAPI_Python)/P1-28 - 비밀번호 찾기/161_auth_password_reset_confirm_route.md
13. ai-notes/backend(fastAPI_Python)/P1-28 - 비밀번호 찾기/162_mail_templates_build_email_reminder_message.md
14. ai-notes/backend(fastAPI_Python)/P1-28 - 비밀번호 찾기/163_password_reset_service_request_email_reminder.md
15. ai-notes/backend(fastAPI_Python)/P1-28 - 비밀번호 찾기/164_auth_email_reminder_request_route.md

백엔드 150 — `hash_reset_token()` (재설정 토큰의 해시)

항목	내용
파일	`studionx/backend/app/core/security.py`
작업	P1-28 비밀번호 찾기 (FR-118): 첫 번째 함수
작성	Claude (AI), 2026-10-14, 사용자 검토
함께 만든 것	모델 `PasswordResetToken`(`app/models/password_reset_token.py`), `User.password_changed_at` 컬럼, 로컬 DB에 migration `2026-10-14_password_reset.sql` 실행
시험	`tests/test_reset_token_hash.py` 12개
다음	토큰 만들기(`generate_reset_token`), 요청 서비스, 새 비밀번호 서비스, 이전 로그인 무효화, 횟수 제한, 라우터 순서로 한 함수씩 진행합니다

1. 하는 일

재설정 링크에 들어가는 토큰 원문을 DB에 저장할 **해시**로 바꿉니다. 원문은 메일 링크에만 있고 DB에는 해시만 있습니다.

입력	결과
`"abc"`	`ba7816bf…0015ad` (소문자 16진수 64자)
같은 문자열을 두 번	항상 같은 값
빈 문자열, `None`, 숫자, `bytes`, 리스트	`ValueError`

2. 코드

```
def hash_reset_token(token: str) -> str:
    if not isinstance(token, str) or token == "":
        raise ValueError("token must be a non-empty string")
    return hashlib.sha256(token.encode("utf-8")).hexdigest()
```

3. 왜 이렇게

선택	이유
bcrypt가 아니라 SHA-256	비밀번호는 사람이 정한 약한 값이라 느린 해시(bcrypt)로 무차별 대입을 늦춰야 합니다. 토큰은 `secrets`로 만든 긴 무작위 값이라 무차별 대입이 불가능하고, 대신 링크를 눌렀을 때 **해시로 행을 바로 찾아야** 합니다. bcrypt는 같은 입력도 매번 다른 값(솔트)이 나와서 찾을 수 없습니다
원문을 저장하지 않음	DB가 유출되어도 해시만으로는 재설정 링크를 만들 수 없습니다
잘못된 값은 `ValueError`	빈 값이나 `bytes`가 조용히 해시되어 저장되는 일을 막습니다. 이 해시는 `token_hash` 유일 색인의 키입니다

4. 함께 한 준비

- **모델** `PasswordResetToken`: `password_reset_tokens` 테이블과 같은 열(`token_hash` 유일, `expires_at`, `used_at`)이고, 회원이 지워지면 함께 지워집니다(`ON DELETE CASCADE`).
- `User.password_changed_at`: 비밀번호를 재설정한 시각입니다. 이 시각보다 먼저 발급된 로그인을 무효로 만드는 데 씁니다(뒤 함수).
- **로컬 DB**: migration을 실행했습니다(이미 들어간 데이터는 그대로이고 열과 테이블만 추가).

5. 시험 (12개 통과)

확인	결과
표준 시험값(`abc`의 SHA-256)	✓
64자 소문자 16진수(`CHAR(64)`에 맞음)	✓
같은 입력은 같은 값, 다른 입력은 다른 값, 대소문자 구분	✓
해시에 원문이 들어 있지 않음	✓
한글은 UTF-8로 해시	✓
빈 문자열, `None`, 숫자, `bytes`, 리스트는 `ValueError`	✓

전체 pytest 696개 통과, `ruff` 통과.

백엔드 151 — `generate_reset_token()` (재설정 토큰 만들기)

항목	내용
파일	`studionx/backend/app/core/security.py`
작업	P1-28 비밀번호 찾기 (FR-118): 두 번째 함수
작성	Claude (AI), 2026-10-14, 사용자 검토
이전	150 `hash_reset_token()`
시험	`tests/test_reset_token_hash.py`의 5개 (이 파일 전체 17개)
다음	요청 서비스(토큰을 만들어 저장하고 메일 링크를 내보냄)

1. 하는 일

새 재설정 토큰을 하나 만들어 **(원문, 해시)** 한 쌍을 돌려줍니다.

값	쓰임
원문 (43자)	메일 링크에만 넣습니다. 어디에도 저장하지 않습니다
해시 (64자)	`password_reset_tokens.token_hash`에 저장합니다

2. 코드

```
RESET_TOKEN_BYTES = 32  # 256비트

def generate_reset_token() -> tuple[str, str]:
    token = secrets.token_urlsafe(RESET_TOKEN_BYTES)
    return token, hash_reset_token(token)
```

3. 왜 이렇게

선택	이유
`secrets.token_urlsafe(32)`	운영체제의 암호용 난수로 무작위 32바이트(256비트)를 만들고, URL에 그대로 넣어도 되는 글자(`A-Z a-z 0-9 - _`) 43자로 바꿉니다. 추측이 불가능한 크기입니다
`random`을 쓰지 않음	`random`은 앞의 값으로 다음 값을 예측할 수 있습니다. 보안 토큰에는 쓰면 안 됩니다
원문과 해시를 한 쌍으로 돌려줌	호출하는 쪽이 해시를 따로 계산하다가 실수하는 일을 막고, 원문을 저장하지 않고 메일에만 쓰는 흐름을 함수 모양으로 드러냅니다
바이트 수를 상수로 뺌	`RESET_TOKEN_BYTES`로 이름을 붙여 시험과 문서가 같은 값을 가리키게 합니다

4. 시험 (5개 추가, 파일 전체 17개 통과)

확인	결과
돌려준 해시가 원문의 `hash_reset_token`과 같음	✓

확인	결과
원문이 43자이고 URL에 안전한 글자만 있음	✓
200번 만들어 원문과 해시가 모두 다름	✓
해시가 64자이고 원문을 포함하지 않음	✓
`secrets`를 32바이트로 불렀음 (`random`이 아님)	✓

전체 pytest 701개 통과, `ruff` 통과.

백엔드 152 — `create_reset_token()` (재설정 토큰 저장)

항목	내용
파일	`studionx/backend/app/services/password_reset_service.py` (새 파일), 설정 `core/config.py`의 `password_reset_minutes`
작업	P1-28 비밀번호 찾기 (FR-118): 세 번째 함수
작성	Claude (AI), 2026-10-14, 사용자 검토
이전	151 `generate_reset_token()`
시험	`tests/test_password_reset_service.py` 7개
다음	`request_password_reset()`: 이메일로 계정을 찾고, 횟수 제한을 검사하고, 이 함수를 부르고, 링크를 로그에 출력

1. 하는 일

회원 한 명의 재설정 토큰을 **새로 만들어 저장**하고, 메일 링크에 쓸 **원문**을 돌려줍니다.

순서	일
1	이 회원의 이전 토큰 중 아직 쓰지 않은 것을 모두 **사용 처리**(`used_at`)합니다
2	`generate_reset_token()`으로 (원문, 해시)를 만들고 **해시만** 저장합니다. 만료는 지금 + 30분입니다
3	`commit`하고 원문을 돌려줍니다. 호출한 쪽이 메일 링크에만 쓰고 저장하지 않습니다

2. 코드

```
def create_reset_token(db: Session, user: User) -> str:
    now = datetime.now(UTC).replace(tzinfo=None)
    expires_at = now + timedelta(minutes=get_settings().password_reset_minutes)

    db.execute(
        update(PasswordResetToken)
        .where(PasswordResetToken.user_id == user.user_id, PasswordResetToken.used_at.is_(None))
        .values(used_at=now)
    )
    token, token_hash = generate_reset_token()
    db.add(PasswordResetToken(user_id=user.user_id, token_hash=token_hash, expires_at=expires_at))
    db.commit()
    return token
```

3. 왜 이렇게

선택	이유
이전 토큰을 사용 처리	다시 요청하면 이전 메일의 링크는 못 쓰게 됩니다. 살아 있는 링크가 항상 하나뿐이라, 오래된 메일이 뒤늦게 악용될 여지가 줄어듭니다
`used_at IS NULL`인 것만 갱신	이미 쓴 토큰의 사용 시각을 덮어쓰지 않습니다(기록 보존)

선택	이유
삭제하지 않고 사용 처리	만료와 사용 기록이 남아서 "이미 쓴 링크입니다"와 "만료된 링크입니다"를 구분해 안내할 수 있습니다. 쌓인 행은 정리 작업(`retention`)에서 지웁니다
한 번에 `commit`	이전 토큰 무효화와 새 토큰 저장이 함께 성공하거나 함께 실패합니다
UTC에서 시간대 정보를 뗀 시각	DB의 `DATETIME`은 시간대가 없고 서버는 UTC로 연결합니다(`session.py`). 다른 서비스와 같은 방식입니다
만료 시간을 설정값으로	`password_reset_minutes`(기본 30)로 두어 시험이나 운영에서 바꿀 수 있습니다
상태와 횟수 제한은 여기서 검사하지 않음	부르는 쪽(요청 서비스)이 먼저 거릅니다. 이 함수는 "토큰을 저장한다"는 한 가지 일만 합니다

4. 시험 (7개 통과)

확인	결과
돌려준 원문의 해시가 저장된 해시와 같고, 원문은 저장되지 않음	✓
만료가 지금 + 30분	✓
새 토큰은 사용하지 않은 상태	✓
두 번째 요청이 첫 번째를 무효로 만들고, 살아 있는 것은 하나뿐	✓
다른 회원의 토큰은 건드리지 않음	✓
이미 쓴 토큰의 사용 시각은 덮어쓰지 않음	✓
5번 요청해도 행 5개, 살아 있는 것은 1개	✓

시험을 확인하는 시험. `used_at=now` 대신 `used_at=None`으로 일부러 틀리게 바꿔 보니 "두 번째 요청이 첫 번째를 무효로 만듦"과 "살아 있는 것은 하나뿐" 두 시험이 실패했고, 원래대로 되돌리자 통과했습니다.

시험에서 고친 것. 만료 시각 시험이 전체 실행에서 한 번 실패했습니다. DB의 `DATETIME(3)`이 밀리초로 반올림해 저장해서, 시험의 기준 시각보다 1밀리초쯤 작게 읽혔기 때문입니다. 함수의 문제가 아니라 시험의 기준이 너무 빡빡했던 것이어서 1초 여유를 두었고, 이후 반복 실행에서 모두 통과했습니다.

전체 pytest 708개 통과, `ruff` 통과.

백엔드 153 — `check_reset_rate_limit()` (재설정 요청 횟수 제한)

항목	내용
파일	`studionx/backend/app/services/password_reset_service.py`
작업	P1-28 비밀번호 찾기 (FR-118): 네 번째 함수
작성	Claude (AI), 2026-10-14, 사용자 검토
이전	152 `create_reset_token()`
시험	`tests/test_password_reset_service.py`의 9개 (이 파일 전체 16개)
다음	`request_password_reset()`: 이메일로 계정을 찾고, 이 함수와 `create_reset_token()`을 부르고, 링크를 서버 로그에 출력

1. 하는 일

재설정 요청이 너무 잦으면 **429**를 내고, 아니면 이번 요청을 세어 둡니다. 한도는 사용자와 합의한 값입니다.

기준	한도 (10분 안)	이유
같은 이메일	3번	한 사람에게 메일이 쏟아지는 것을 막습니다
같은 IP	10번	같은 망을 쓰는 여러 사람을 감안해 넉넉하게 둡니다

한도를 넘으면 `AppError(TOO_MANY_REQUESTS, 429, "Too many requests. Please try again later.")`이고, `details.retry_after`에 몇 초 뒤에 다시 할 수 있는지를 담습니다.

2. 코드 (요약)

```
RESET_LIMIT_SECONDS = 10 * 60
RESET_LIMIT_EMAIL = 3
RESET_LIMIT_IP = 10
_recent_requests: dict[str, deque[float]] = {}

def check_reset_rate_limit(ip: str | None, email: str) -> None:
    now = _clock()
    keys = [(f"email:{email.strip().lower()}", RESET_LIMIT_EMAIL)]
    if ip:
        keys.append((f"ip:{ip}", RESET_LIMIT_IP))
    with _lock:
        retry_after = 0
        for key, limit in keys:
            q = _recent_requests.get(key)
            if q is None:
                continue
            while q and now - q[0] > RESET_LIMIT_SECONDS:
                q.popleft()                     # 10분이 지난 기록은 버림
            if not q:
                del _recent_requests[key]       # 비면 열쇠째 지움
                continue
            if len(q) >= limit:
                retry_after = max(retry_after, int(RESET_LIMIT_SECONDS - (now - q[0])) + 1)
```

```
        if retry_after:
            raise AppError(ErrorCode.TOO_MANY_REQUESTS, 429,
                           "Too many requests. Please try again later.",
                           details={"retry_after": retry_after})
        for key, _ in keys:
            _recent_requests.setdefault(key, deque()).append(now)
```

3. 왜 이렇게

선택	이유
가입 여부와 관계없이 모든 요청을 셈	없는 이메일만 제한에 안 걸리거나 먼저 걸리면 응답 차이로 가입 여부를 알아낼 수 있습니다. FR-118의 "항상 같은 응답"을 지키기 위해서입니다
문의와 달리 **검사와 세기를 한 함수로**	문의는 저장에 실패한 시도를 세지 않으려고 둘로 나눴습니다. 여기서는 가입 여부를 숨겨야 해서 항상 세므로 나눌 이유가 없습니다
한도를 넘은 요청은 세지 않음	계속 눌러도 제한 시간이 늘어나지 않고, `retry_after`가 시간이 지나며 줄어듭니다
이메일은 소문자와 공백 제거	`A@Example.com`과 `a@example.com`이 다른 사람으로 세어지면 제한을 피할 수 있습니다
둘 중 하나라도 넘으면 막음	이메일 기준과 IP 기준을 모두 검사하고, `retry_after`는 둘 중 긴 쪽입니다. 막히지 않았을 때만 둘을 함께 셉니다
메모리 방식	문의(FR-113)와 같은 결정입니다. 서버를 다시 켜면 비워지고, 서버가 여러 대면 따로 셉니다. 정식 운영에서 서버를 늘리면 Redis 같은 곳으로 옮겨야 합니다
오래된 기록 정리	열쇠가 비면 지워서, 여러 이메일로 요청해도 메모리가 계속 늘지 않습니다
시계를 `_clock`으로 분리	시험에서 시간을 직접 움직입니다(`time.monotonic`: 시스템 시계를 바꿔도 영향이 없음)

4. 시험 (9개 추가, 파일 전체 16개 통과)

확인	결과
같은 이메일은 3번 통과하고 4번째에 429	✓
이메일은 대소문자와 앞뒤 공백을 무시	✓
같은 IP는 이메일이 달라도 10번까지, 11번째에 429	✓
다른 이메일과 다른 IP는 영향 없음	✓
막힌 요청은 세지 않고, `retry_after`가 시간이 지나며 줄어듦	✓
10분이 지나면 다시 통과	✓
가입된 이메일과 없는 이메일이 똑같이 3번 통과하고 4번째에 막힘	✓
IP가 없으면 이메일 기준만 적용	✓
만료된 기록이 정리되어 메모리가 늘지 않음	✓

시험을 확인하는 시험. 일부러 틀리게 바꿔 시험이 잡는지 보았습니다.

- 한도 비교를 `>=`에서 `>`로 바꾸자 6개가 실패했습니다.
- 막힌 요청도 세도록 바꾸자 9개가 실패했습니다.
- 둘 다 원래대로 되돌린 뒤 통과했습니다.

전체 pytest 717개 통과, `ruff` 통과.

백엔드 154 — `request_password_reset()` (재설정 요청)

항목	내용
파일	`studionx/backend/app/services/password_reset_service.py`, 설정 `core/config.py`(`mail_backend`, `app_base_url`), `studionx/.env.example`
작업	P1-28 비밀번호 찾기 (FR-118): 다섯 번째 함수 (`mask_email()` 도우미 포함)
작성	Claude (AI), 2026-10-14, 사용자 검토
이전	153 `check_reset_rate_limit()`
시험	`tests/test_password_reset_service.py`의 13개 (이 파일 전체 29개)
다음	`reset_password()`: 토큰과 새 비밀번호로 비밀번호를 바꾸고 이전 로그인을 무효로 만듦

1. 하는 일

재설정 요청을 처리합니다. **가입 여부와 관계없이 항상 같은 결과(`None`)**를 돌려줘서, 응답으로 가입 여부를 알아낼 수 없습니다.

순서	일
1	`check_reset_rate_limit()`으로 횟수를 검사합니다. 넘으면 429입니다
2	이메일(소문자, 공백 제거)로 회원을 찾습니다. 없거나 `ACTIVE`가 아니면(탈퇴, 정지) 아무것도 하지 않고 끝냅니다
3	`ACTIVE`이면 `create_reset_token()`으로 토큰을 만들고, 링크를 메일(지금은 서버 로그)로 보냅니다

링크의 모양은 `http://localhost:8000/resetPassword.html#token=<토큰 원문>`입니다.

2. 코드 (요약)

```
def request_password_reset(db: Session, email: str, ip: str | None) -> None:
    check_reset_rate_limit(ip, email)

    user = db.scalar(select(User).where(User.email == email.strip().lower()))
    if user is None or user.status != UserStatus.ACTIVE:
        return

    settings = get_settings()
    if settings.mail_backend != "log":
        logger.warning("password reset requested but mail is not configured (MAIL_BACKEND)")
        return

    token = create_reset_token(db, user)
    link = f"{settings.app_base_url.rstrip('/')}/resetPassword.html#token={token}"
    logger.info("[DEV MAIL] password reset link for %s: %s", mask_email(user.email), link)
```

3. 개인정보와 보안 판단

사용자가 "개인정보적으로 문제가 없으면 그렇게 하라"고 하셔서 문제 지점을 따져 보고 아래와 같이 막았습니다.

우려	처리
토큰이 접속 기록과 Referer로 새는 것	토큰을 `?token=`이 아니라 `#token=` 뒤에 둡니다. `#` 뒤의 값은 브라우저가 서버로 보내지 않아서 서버 접속 기록, 프록시 기록, 다른 사이트로 가는 `Referer`에 남지 않습니다
로그에 이메일이 남는 것	이메일을 가려서 남깁니다(`real@example.com` → `r***@example.com`)
로그에 링크가 남는 것 (링크 = 비밀번호를 바꿀 수 있는 값)	**개발용(`MAIL_BACKEND=log`)일 때만** 출력합니다. 그 밖의 설정이면 토큰도 링크도 만들지 않고 "메일 설정 없음"만 기록합니다. 배포에서 실수로 이 방식이 켜져 있어도 `log`가 아니면 링크가 남지 않습니다
응답으로 가입 여부가 드러나는 것	항상 같은 결과를 돌려주고, 횟수 제한도 가입 여부와 관계없이 셉니다
탈퇴, 정지 계정	가입된 주소여도 조용히 아무것도 하지 않습니다(`ACTIVE`만 처리)
링크가 오래 살아 있는 것	30분 유효, 한 번만 사용, 다시 요청하면 이전 링크가 무효

배포 전 남은 일(문서에 남깁니다).

1. SMTP 같은 실제 메일 발송을 붙이고 `MAIL_BACKEND`를 그 값으로 바꿔야 합니다. 그 전에는 배포에서 재설정이 동작하지 않습니다(안전한 쪽으로 실패).
2. `resetPassword.html`은 `#token=`을 읽은 뒤 주소창에서 지우고(`history.replaceState`) `Referrer-Policy: no-referrer`를 두어야 합니다. 화면 작업 때 반영합니다.
3. 개인정보 처리방침(`privacy.html`, 세 판)에 "비밀번호 재설정 메일 발송"과 처리 항목을 반영하는 일은 정식 공개 전 검토 때 함께 합니다.

4. 시험 (13개 추가, 파일 전체 29개 통과)

확인	결과
`ACTIVE` 회원은 로그에 링크가 나오고 해시가 저장됨, 링크의 `/`가 겹치지 않음	✓
토큰이 `?`가 아니라 `#` 뒤에 있음	✓
로그에 이메일이 가려져 있고 원문 이메일은 없음	✓
이메일의 대소문자와 공백을 무시하고 찾음	✓
없는 이메일은 아무것도 만들지 않고 같은 결과	✓
탈퇴, 정지 회원은 아무것도 만들지 않고 같은 결과 (2개)	✓ ✓
없는 이메일에도 횟수 제한이 똑같이 적용됨	✓
두 번째 요청이 첫 링크를 무효로 만듦	✓
`MAIL_BACKEND`가 `log`가 아니면 토큰도 링크도 만들지 않음	✓
`mask_email` (3가지)	✓

시험을 확인하는 시험. 일부러 틀리게 바꿔 시험이 잡는지 보았습니다.

- 상태 검사를 지우자 2개 실패
- `#token=`을 `?token=`으로 바꾸자 2개 실패
- 이메일 가리기를 지우자 1개 실패

- `mail_backend` 검사를 지우자 1개 실패
- 모두 되돌린 뒤 통과했습니다.

전체 pytest 730개 통과, `ruff` 통과.

백엔드 155 — `reset_password()` (새 비밀번호로 바꾸기)

항목	내용
파일	`studionx/backend/app/services/password_reset_service.py`, 오류 코드 `core/errors.py`의 `INVALID_RESET_LINK`
작업	P1-28 비밀번호 찾기 (FR-118): 여섯 번째 함수
작성	Claude (AI), 2026-10-14, 사용자 검토
이전	154 `request_password_reset()`
시험	`tests/test_password_reset_service.py`의 10개 (이 파일 전체 39개)
다음	이전 로그인 무효화: 토큰의 발급 시각(`iat`)이 `password_changed_at`보다 앞서면 거부

1. 하는 일

메일 링크의 토큰과 새 비밀번호를 받아 비밀번호를 바꿉니다.

순서	일	실패하면
1	토큰의 해시로 행을 찾고, 만료, 사용 여부, 회원 상태(`ACTIVE`)를 확인합니다	400 `INVALID_RESET_LINK` ("This link is invalid or has expired.")
2	새 비밀번호 규칙(FR-102)을 검사합니다	422 + `details.new_password`. **토큰은 쓰지 않은 채로 둡니다**
3	토큰을 한 번만 쓰도록 사용 처리합니다(`UPDATE … WHERE used_at IS NULL`, 바뀐 행이 1이어야 함)	400 `INVALID_RESET_LINK`
4	새 해시 저장, `password_changed_at` 기록, 이 회원의 남은 토큰도 모두 사용 처리, `commit`	

2. 코드 (요약)

```
def reset_password(db: Session, token: str, new_password: str) -> None:
    try:
        token_hash = hash_reset_token(token)
    except ValueError:
        raise _invalid_link() from None

    now = datetime.now(UTC).replace(tzinfo=None)
    row = db.scalar(select(PasswordResetToken).where(PasswordResetToken.token_hash == token_hash))
    if row is None or row.used_at is not None or row.expires_at <= now:
        raise _invalid_link()
    user = db.get(User, row.user_id)
    if user is None or user.status != UserStatus.ACTIVE:
        raise _invalid_link()

    failed = check_password_rules(new_password)
    if failed:
        raise AppError(ErrorCode.VALIDATION_ERROR, 422, "Check the password requirements.",
                       details={"new_password": failed})
```

```
    claimed = db.execute(update(PasswordResetToken)
        .where(PasswordResetToken.token_id == row.token_id, PasswordResetToken.used_at.is_(None))
        .values(used_at=now))
    if claimed.rowcount != 1:
        db.rollback()
        raise _invalid_link()

    user.password_hash = hash_password(new_password)
    user.password_changed_at = now
    db.execute(update(PasswordResetToken)
        .where(PasswordResetToken.user_id == user.user_id, PasswordResetToken.used_at.is_(None))
        .values(used_at=now))
    db.commit()
```

3. 왜 이렇게

선택	이유
틀림, 만료, 이미 씀, 탈퇴 계정이 모두 같은 오류	이유를 알려 주면 "이 토큰은 한때 유효했다" 같은 정보를 추측하게 됩니다. 사용자에게는 "링크가 잘못되었거나 만료되었습니다. 다시 요청하세요"로 충분합니다
링크 검사가 비밀번호 규칙 검사보다 먼저	링크가 틀렸으면 비밀번호가 약해도 링크 오류가 먼저 나와야, 링크 문제가 가려지지 않습니다
비밀번호 규칙이 틀려도 토큰은 쓰지 않음	사용자가 화면에서 비밀번호만 고쳐서 다시 보낼 수 있습니다. 토큰을 먼저 써 버리면 약한 비밀번호 한 번에 링크가 죽어서 다시 메일을 받아야 합니다
`UPDATE … WHERE used_at IS NULL`로 한 번만 쓰기	조회(`SELECT`)와 사용 처리 사이에 같은 링크를 동시에 누른 두 요청이 모두 통과하는 틈이 있습니다. 조건부 `UPDATE`는 DB가 한 번에 처리해서 하나만 1행을 바꾸고, 나머지는 0행이라 막힙니다
이 회원의 남은 토큰도 모두 사용 처리	비밀번호를 바꿨는데 이전에 받은 다른 링크가 살아 있으면 그것으로 또 바꿀 수 있습니다
`password_changed_at` 기록	이 시각보다 먼저 발급된 로그인(액세스, 리프레시 토큰)을 무효로 만드는 데 씁니다. 그 검사는 다음 함수입니다
현재 비밀번호와 같은 값을 막지 않음	비밀번호 변경(FR-107)은 같은 값을 막지만, 재설정은 잊어서 오는 흐름이라 의미가 없습니다
새 오류 코드 `INVALID_RESET_LINK`	화면(JS)이 이 코드를 보고 "다시 요청" 안내로 분기합니다

4. 시험 (10개 추가, 파일 전체 39개 통과)

확인	결과
유효한 토큰으로 비밀번호가 바뀌고, 예전 비밀번호는 못 쓰고, `password_changed_at`이 기록됨	✓
토큰은 한 번만 쓸 수 있음(두 번째는 400, 반영 안 됨)	✓
없는 토큰, 만료, 이미 씀, 빈 값이 **같은 오류, 같은 문구**	✓
약한 비밀번호는 422이고 토큰, 비밀번호가 그대로이며 고쳐서 다시 보내면 됨	✓
링크가 틀리면 비밀번호가 약해도 링크 오류가 먼저	✓
탈퇴, 정지 회원은 재설정 불가 (2개)	✓ ✓
이 회원의 다른 살아 있는 토큰이 모두 무효가 됨	✓
다른 회원은 영향을 받지 않음	✓
조회 뒤 다른 요청이 먼저 토큰을 쓴 경우(경쟁): 같은 오류, 비밀번호 그대로	✓

시험을 확인하는 시험. 일부러 틀리게 바꿔 시험이 잡는지 보았습니다.

바꾼 것	결과
만료 검사 제거	1개 실패
`password_changed_at` 기록 제거	1개 실패
남은 토큰 무효화 제거	1개 실패
상태(`ACTIVE`) 검사 제거	2개 실패
사용 처리 행 수(`rowcount`) 확인 제거	1개 실패
"이미 씀" 검사 제거	**실패 없음**

마지막은 시험의 빈틈이 아니라 **중복 방어**입니다. "이미 씀" 검사를 지워도 `UPDATE … WHERE used_at IS NULL`의 행 수 확인이 같은 요청을 막아서 동작이 같기 때문입니다. 두 검사를 함께 둔 것은 첫 검사가 빠른 거절(쓸데없이 비밀번호 규칙 검사와 해시를 하지 않음)을, 두 번째가 경쟁 상태의 안전망을 맡도록 한 것입니다.

전체 pytest 740개 통과, `ruff` 통과.

5. 다음 함수에 남기는 주의

`password_changed_at`은 밀리초까지 저장하지만 JWT의 발급 시각(`iat`)은 **초 단위**입니다. 비교할 때 초 미만을 버리지 않으면, 재설정 직후 같은 초 안에 새로 로그인한 토큰이 "이전 로그인"으로 잘못 거부될 수 있습니다. 다음 함수에서 이 경계를 시험으로 확인합니다.

백엔드 156 — `issued_before_password_change()` (이전 로그인 무효 판단)

항목	내용
파일	`studionx/backend/app/core/security.py`
작업	P1-28 비밀번호 찾기 (FR-118): 일곱 번째 함수
작성	Claude (AI), 2026-10-15, 사용자 검토
이전	155 `reset_password()`
시험	`tests/test_reset_token_hash.py`의 14개 (이 파일 전체 31개)
다음	이 판단을 `get_current_user`(액세스 토큰)와 `/auth/refresh`(리프레시 토큰)에 연결

1. 하는 일

로그인 토큰이 비밀번호를 재설정하기 **전에** 발급된 것이면 `True`(= 무효)를 돌려주는 **판단 함수**입니다. 비밀번호를 잊어서 재설정한다면 그 계정에 이미 들어와 있던 다른 사람(또는 도난당한 토큰)을 내보내는 것이 목적입니다.

입력	결과
재설정한 적이 없음(`password_changed_at`이 `None`)	항상 `False` (기존 로그인은 그대로)
`iat`가 재설정 시각의 초보다 **앞**	`True` (무효)
`iat`가 재설정 시각의 초와 **같거나 뒤**	`False` (유효)
재설정했는데 `iat`가 없거나 숫자가 아님	`True` (확인할 수 없으면 거부)

2. 코드

```
def issued_before_password_change(issued_at: object, password_changed_at: datetime | None) -> bool:
    if password_changed_at is None:
        return False
    if isinstance(issued_at, bool) or not isinstance(issued_at, int | float):
        return True
    changed = calendar.timegm(password_changed_at.utctimetuple())
    return issued_at < changed
```

3. 왜 이렇게

선택	이유
초 단위로 비교	JWT의 `iat`는 초 단위이고 `password_changed_at`은 밀리초까지 있습니다. 그대로 비교하면 재설정 직후 **같은 초**에 새로 로그인한 사람의 토큰이 "이전 것"으로 잘못 거부됩니다. 그래서 `password_changed_at`의 초 미만을 버리고 `iat < 그 초`일 때만 무효로 봅니다
같은 초의 토큰은 **유효**	반대쪽 위험(재설정과 같은 초에 앞서 발급된 옛 토큰이 살아남음)은 현실적으로 없습니다. 재설정은 메일 링크를 거쳐서, 같은 초에 앞선 로그인이 있을 수 없습니다

선택	이유
`calendar.timegm(….utctimetuple())`	DB의 시각은 UTC이고 시간대 정보가 없습니다. `datetime.timestamp()`는 시간대 정보가 없는 값을 **서버의 로컬 시간**으로 읽어서 서버 시간대에 따라 결과가 달라집니다. `timegm`은 항상 UTC로 읽습니다
이상한 `iat`는 무효	재설정한 계정에서 `iat`가 없거나 문자열이거나 `True`인 토큰은 우리가 만든 토큰이 아닙니다. 확인할 수 없으면 거부하는 쪽이 안전합니다. `bool`은 `int`의 하위 형이라 따로 걸러야 합니다
순수 함수로 분리	DB나 토큰 해독 없이 시각 비교만 해서 경계 시험을 정확히 할 수 있습니다. 실제 연결은 다음 함수가 합니다

4. 시험 (14개 추가, 파일 전체 31개 통과)

확인	결과
재설정한 적이 없으면 항상 유효(`iat`가 이상해도)	✓
재설정 이전 발급은 무효	✓
재설정 이후 발급은 유효	✓
같은 초 발급은 유효(밀리초 .000, .500, .999 모두)	✓
초 경계 바로 앞은 무효, 바로 위는 유효	✓
소수 `iat`는 값으로 비교	✓
`None`, 문자열, `True`, `False`, 리스트, 딕셔너리, `bytes`인 `iat`는 무효 (7개)	✓
UTC로 읽음(서버 시간대와 무관)	✓

시험을 확인하는 시험. 일부러 틀리게 바꿔 시험이 잡는지 보았습니다.

바꾼 것	결과
`<`를 `<=`로	2개 실패
초 미만을 버리지 않고 밀리초까지 비교	1개 실패 (**같은 초의 새 로그인이 거부되는 오류**를 잡음)
`None`일 때의 빠른 반환 제거	1개 실패
이상한 `iat`를 유효로	7개 실패
`timegm` 대신 로컬 시간(`timestamp()`)	4개 실패
`bool` 허용	**실패 없음**

마지막은 `True`(=1)를 `iat`로 받아도 `1 < 재설정 시각`이라 어차피 `True`(무효)가 되어 결과가 같기 때문입니다. `bool` 검사는 `iat=True`가 "유효한 시각"처럼 취급되지 않게 하는 방어이고, 동작을 가르는 경계가 없어 시험으로 잡을 수 없는 중복 방어입니다.

전체 pytest 754개 통과, `ruff` 통과.

5. 다음 함수에 남기는 일

이 판단을 쓰려면 토큰에서 `iat`를 꺼내야 하는데, 지금 `decode_access_token`과 `decode_refresh_token`은 `user_id`만 돌려줍니다. 다음 함수에서 다음 중 하나로 연결해야 합니다.

1. `iat`를 함께 돌려주는 새 해독 함수를 더하고(기존 함수와 시험은 그대로), `get_current_user`와 `/auth/refresh`가 그것을 씁니다.
2. 기존 함수의 반환값을 `(user_id, iat)`로 바꿉니다. 호출하는 곳과 시험이 많이 바뀝니다.

1번을 권합니다. 기존 시험과 호출부를 건드리지 않기 때문입니다.

백엔드 157 — `decode_token_with_iat()` (토큰에서 user_id와 발급 시각 꺼내기)

항목	내용
파일	`studionx/backend/app/core/security.py`
작업	P1-28 비밀번호 찾기 (FR-118): 여덟 번째 함수 (내부 도우미 `_decode_claims()` 포함)
작성	Claude (AI), 2026-10-15, 사용자 검토
이전	156 `issued_before_password_change()`
시험	`tests/test_security.py`의 14개 (이 파일 전체 45개)
다음	`get_current_user`(액세스 토큰)와 `/auth/refresh`(리프레시 토큰)에 연결

1. 하는 일

토큰을 검사하고 **(user_id, 발급 시각 `iat`)** 를 돌려줍니다. 156번의 `issued_before_password_change()`가 `iat`를 필요로 하는데, 기존 `decode_access_token()`과 `decode_refresh_token()`은 `user_id`만 돌려주기 때문에 필요한 함수입니다.

입력	결과
정상 액세스 토큰, `"access"`	`(7, 1760443200)` 같은 (user_id, iat)
정상 리프레시 토큰, `"refresh"`	같은 모양
`iat`가 없는 토큰	`(7, None)` (형식 판단은 쓰는 쪽 몫)
서명, 만료, 형식, 종류가 틀림	기존과 같은 401 "Please sign in again."
`token_type`이 `"access"`, `"refresh"`가 아님	`ValueError` (부르는 쪽의 실수)

2. 코드

```
def _decode_token(token: str, expected_type: str) -> int:
    return _decode_claims(token, expected_type)[0]          # 기존 동작 그대로

def _decode_claims(token: str, expected_type: str) -> tuple[int, object]:
    ...   # 기존 _decode_token의 검사와 같고, (int(payload["sub"]), payload.get("iat"))를 돌려줌

def decode_token_with_iat(token: str, token_type: str) -> tuple[int, object]:
    if token_type not in ("access", "refresh"):
        raise ValueError("token_type must be 'access' or 'refresh'")
    return _decode_claims(token, token_type)
```

3. 왜 이렇게

선택	이유
기존 함수의 반환값을 바꾸지 않고 **새 함수를 더함**	`decode_access_token`을 부르는 곳과 시험이 많습니다. 반환값을 `(user_id, iat)`로 바꾸면 모두 고쳐야 하고, 인증의 핵심 경로라서 위험합니다
검사 코드를 `_decode_claims()` 한 곳으로 모음	기존 함수와 새 함수가 **같은 코드**로 검사해서 서로 어긋날 수 없습니다. 같은 검사를 복사해 두면 한쪽만 고치는 실수가 생깁니다
`_decode_token()`은 껍데기로 남김	이름과 동작이 그대로라 기존 호출부와 시험이 바뀌지 않습니다
`iat`를 적힌 그대로 돌려줌	형식 검사(숫자인지, 없는지)는 `issued_before_password_change()`가 이미 합니다. 한 곳에서만 판단합니다
잘못된 `token_type`은 `ValueError`	`"ACCESS"`처럼 잘못 쓰면 모든 토큰이 401로 거부되어 원인을 찾기 어렵습니다. 부르는 쪽의 실수는 바로 알립니다

4. 시험 (14개 추가, 파일 전체 45개 통과)

확인	결과
진짜 액세스 토큰에서 user_id와 정수 `iat`를 꺼냄	✓
리프레시 토큰도 같음	✓
`iat`가 없으면 `None`	✓
토큰에 적힌 `iat`를 그대로 돌려줌	✓
기존 `decode_access_token`, `decode_refresh_token`과 같은 user_id	✓
틀린 토큰 7종(깨진 글자, 빈 문자열, 틀린 비밀키, 만료, `sub` 없음, `sub`가 숫자 아님, 종류가 다름)이 **같은 401, 같은 문구**	✓ (7개)
액세스를 리프레시로, 리프레시를 액세스로 물으면 거부	✓
`alg: none` 위조 토큰 거부	✓
잘못된 `token_type`(`""`, `"ACCESS"`, `"id"`, `None`, `1`)은 `ValueError`	✓ (5개)
156번 함수와 이어 쓰기: 같은 초는 유효, 재설정이 나중이면 무효, 재설정한 적 없으면 유효	✓

시험을 확인하는 시험. 일부러 틀리게 바꿔 시험이 잡는지 보았습니다.

바꾼 것	결과
`iat`를 항상 `None`으로	4개 실패
토큰 종류 검사 제거	5개 실패
`token_type` 검사 제거	5개 실패
기존 함수가 다른 값을 돌려주게	4개 실패
알고리즘 고정 해제(`["HS256", "none"]`)	**실패 없음**

마지막은 시험의 빈틈이 아니라 **라이브러리의 이중 방어**입니다. PyJWT는 키가 있는 상태에서 `alg: none`을 풀려고 하면 스스로 오류를 내서, 알고리즘 목록에 `none`을 넣어도 위조 토큰이 거부됩니다. 이 코드의 `algorithms=[JWT_ALGORITHM]` 고정은 그 라이브러리 동작에 기대지 않고 의도를 코드에 명시하는 것입니다.

시험을 만들며 고친 것. 새 시험이 짧은 비밀키(12바이트)를 써서 PyJWT의 키 길이 경고가 한 번 났습니다. 기존 시험처럼 32바이트 이상의 키로 바꿔서 경고를 없앴습니다.

전체 pytest 774개 통과, `ruff` 통과.

백엔드 158 — `get_current_user()`에 재설정 이전 로그인 거부 연결

항목	내용
파일	`studionx/backend/app/api/deps.py`의 `get_current_user()`
작업	P1-28 비밀번호 찾기 (FR-118): 아홉 번째 함수 (기존 함수에 검사 하나를 더함)
작성	Claude (AI), 2026-10-15, 사용자 검토
이전	157 `decode_token_with_iat()`
시험	`tests/test_deps.py` 6개 추가(파일 전체 19개), `tests/test_password_reset_service.py` 1개 추가
다음	`/auth/refresh`(리프레시 토큰)에 같은 검사를 연결

1. 하는 일

로그인이 필요한 모든 API가 거치는 `get_current_user()`에서, 비밀번호를 재설정하기 **전에** 발급된 액세스 토큰을 **401**로 거부합니다. 그 계정에 이미 로그인해 있던 다른 기기나 도난당한 토큰이 재설정 뒤에는 쓸 수 없게 됩니다.

상황	결과
재설정한 적 없는 계정의 토큰	그대로 통과 (기존 로그인은 영향 없음)
재설정 **이전**에 발급된 토큰	401 "Please sign in again." (다른 401과 같은 문구)
재설정 **이후**(같은 초 포함)에 발급된 토큰	통과
재설정한 계정인데 `iat`가 없는 토큰	401

2. 바뀐 코드

```
user_id, issued_at = decode_token_with_iat(credentials.credentials, "access")   # decode_access_token에서 교체
user = db.get(User, user_id)
if user is None or user.status != "ACTIVE":
    raise AppError(ErrorCode.UNAUTHORIZED, 401, "Please sign in again.")
if issued_before_password_change(issued_at, user.password_changed_at):          # 새로 더한 검사
    raise AppError(ErrorCode.UNAUTHORIZED, 401, "Please sign in again.")
return user
```

3. 왜 이렇게

선택	이유
같은 401, 같은 문구	다른 이유(만료, 없는 회원)와 구분되지 않게 해서, 응답으로 "비밀번호가 바뀌었다"는 사실을 알아낼 수 없습니다
회원 상태 검사 **뒤**에 둠	회원이 없거나 탈퇴 계정이면 `user.password_changed_at`을 읽기 전에 거부됩니다
`get_current_user_optional()`은 **고치지 않음**	이 함수가 `get_current_user()`를 부르고 실패하면 손님으로 처리하므로, 무효가 된 토큰은 자동으로 손님이 됩니다(시험으로 확인)

선택	이유
401이면 화면이 재발급을 시도하는 점	`api.js`는 401을 받으면 `/auth/refresh`로 토큰을 다시 받으려 합니다. 리프레시 토큰도 **다음 함수**에서 같은 검사로 거부해야 합니다. 그 전에는 재발급이 성공해 이전 기기가 계속 로그인된 채로 남습니다

중요: 이 함수만으로는 아직 완전하지 않습니다. 액세스 토큰은 30분이면 만료되지만 리프레시 토큰은 3일 동안 새 액세스 토큰을 받아 갑니다. `/auth/refresh`를 고치기 전까지 이전 기기는 재발급으로 로그인을 이어 갈 수 있습니다. 다음 함수에서 닫습니다.

4. 시험

`test_deps.py` **6개 추가**(파일 전체 19개)

확인	결과
재설정 이전에 발급된 토큰은 같은 401	✓
재설정 이후에 발급된 토큰은 통과	✓
재설정과 같은 초에 발급된 토큰은 통과(재설정 직후 새 로그인이 쫓겨나지 않음)	✓
재설정한 계정에서 `iat`가 없는 토큰은 401	✓
재설정한 적 없는 계정은 `iat`가 없어도 통과	✓
`get_current_user_optional`은 무효 토큰을 손님으로 처리	✓

실제 앱과 실제 MySQL로 한 통합 시험 1개(`test_password_reset_service.py`)

1. 한 시간 전에 발급된 토큰으로 `/api/v1/me/dashboard` 요청 → 200 (재설정 전)
2. `reset_password()` 실행
3. 같은 토큰으로 다시 요청 → **401**
4. 재설정 뒤 새로 발급한 토큰으로 요청 → **200**

기존 시험의 가짜 회원(`SimpleNamespace`)에는 `password_changed_at=None`을 더했습니다. 새 검사가 이 칸을 읽기 때문입니다.

시험을 확인하는 시험. 일부러 틀리게 바꿔 시험이 잡는지 보았습니다.

바꾼 것	결과
재설정 검사 제거	3개 실패
검사 결과를 반대로	8개 실패
`iat` 대신 `None`을 넘김	2개 실패
액세스 토큰을 `"refresh"` 종류로 해독	5개 실패

전체 pytest 781개 통과, `ruff` 통과.

5. 시험을 고치며 있었던 일

줄 길이를 맞추려고 줄 번호로 `sed`를 돌렸다가, 번호가 어긋나서 엉뚱한 줄(시험의 구역 설명 주석)을 덮어쓴 것을 `grep`으로 발견해 복구했습니다. 156번 작업에서 같은 실수를 한 뒤 두 번째입니다. 이후로는 줄 번호가 아니라 **내용으로 찾아 바꾸는** 방식(`str.replace`)만 씁니다. 최종 시험은 그대로이고 복구 뒤 전체 시험을 다시 돌려 통과를 확인했습니다.

백엔드 159 — `/auth/refresh`에 재설정 이전 로그인 거부 연결

항목	내용
파일	`studionx/backend/app/api/v1/auth.py`의 `refresh()`
작업	P1-28 비밀번호 찾기 (FR-118): 열 번째 함수 (기존 함수에 검사 하나를 더함)
작성	Claude (AI), 2026-10-15, 사용자 검토
이전	158 get_current_user()
시험	`tests/test_auth_routes.py` 4개 추가 (파일 전체 47개)
다음	라우터: 재설정 요청과 새 비밀번호 API 두 개를 `/api/v1/auth`에 연결

1. 하는 일

`/auth/refresh`(액세스 토큰 재발급)에서, 비밀번호를 재설정하기 **전에** 발급된 리프레시 토큰을 **401로 거부하고 쿠키도 지웁니다.** 158번과 함께 이전 기기가 로그인을 이어 가는 두 길(액세스 토큰, 재발급)이 모두 닫힙니다.

상황	결과
재설정한 적 없는 계정	전과 같이 200 (쿠키는 건드리지 않음)
재설정 **이전**에 발급된 리프레시 토큰	401 "Please sign in again." + 쿠키 삭제
재설정 **이후**(같은 초 포함)에 새로 로그인한 토큰	200

2. 바뀐 코드

```
user_id, issued_at = decode_token_with_iat(refresh_token, "refresh")   # decode_refresh_token에서 교체
user = auth_service.get_active_user(db, user_id)
if issued_before_password_change(issued_at, user.password_changed_at):   # 새로 더한 검사
    stale = error_json(401, ErrorCode.UNAUTHORIZED, "Please sign in again.")
    clear_refresh_cookie(stale)
    return stale
return TokenResponse(...)
```

3. 왜 이렇게

선택	이유
`AppError`를 던지지 않고 401 응답을 직접 돌려줌	`AppError`를 던지면 예외 처리기가 **새 응답**을 만들어서, 앞서 쿠키 삭제를 적어 둔 헤더가 버려집니다. 쿠키를 지우려면 응답 객체에 직접 적어서 돌려줘야 합니다(시험으로 확인: 던지는 방식으로 바꾸면 쿠키 삭제 시험이 실패)
응답 모양은 `error_json`으로 같게	다른 401과 같은 JSON(`code`, `message`, `details`)이라 화면(`api.js`)의 처리가 그대로 동작합니다
쿠키를 함께 지움	이미 무효가 된 토큰을 브라우저가 `/auth` 아래 요청마다 계속 보내는 것을 막습니다. 지운 뒤에는 "쿠키 없음"(정상 흐름의 401)이 됩니다
같은 문구 "Please sign in again."	만료나 위조된 토큰과 구분되지 않아서 응답으로 "비밀번호가 바뀌었다"를 알아낼 수 없습니다

선택	이유
회원 상태 검사(`get_active_user`) 뒤에 둠	탈퇴, 정지 계정은 `password_changed_at`을 읽기 전에 거부됩니다

4. 시험 (4개 추가, 파일 전체 47개 통과, 실제 앱과 실제 MySQL)

확인	결과
로그인 뒤 재설정하면 재발급이 401, 본문이 다른 401과 같고, `Set-Cookie`로 쿠키가 지워짐(`Max-Age=0`, 같은 경로, `HttpOnly`)	✓
쿠키가 지워진 뒤의 요청은 "쿠키 없음"(정상 흐름의 401, 문구 "Please sign in.")	✓
재설정 직후(같은 초일 수 있음) 새로 로그인하면 재발급이 200	✓
재설정한 적 없는 계정의 재발급은 전과 같음(200, `Set-Cookie` 없음)	✓

시험을 확인하는 시험. 일부러 틀리게 바꿔 시험이 잡는지 보았습니다.

바꾼 것	결과
재설정 검사 제거	2개 실패
쿠키 삭제 제거	2개 실패
`iat` 대신 `None`을 넘김	1개 실패
`AppError`를 던지는 방식으로 되돌림(쿠키 삭제 유실)	2개 실패
리프레시 토큰을 `"access"` 종류로 해독	8개 실패

기존 `refresh` 시험(재발급 성공, 쿠키 없음, 엉터리 토큰, 탈퇴 뒤, 로그아웃 뒤)은 모두 그대로 통과합니다.

전체 pytest 785개 통과, `ruff` 통과.

5. 이제 닫힌 것

경로	이전 기기의 로그인	닫은 함수
액세스 토큰(30분)	재설정 뒤 401	158
리프레시 토큰(3일)으로 재발급	재설정 뒤 401 + 쿠키 삭제	159

이전 기기는 재설정 뒤 최대 몇 분 안에(다음 요청에서) 로그인 화면으로 돌아갑니다. 새 비밀번호로 다시 로그인한 기기는 영향이 없습니다.

6. 남은 일

- 비밀번호 **변경**(FR-107, 로그인한 사용자가 현재 비밀번호를 알고 바꾸는 경우)에는 이 검사를 적용하지 않았습니다. 그 기능은 `password_changed_at`을 기록하지 않아서 다른 기기가 로그아웃되지 않습니다(`user_service.change_password`의 기존 결정: "다른 기기 로그아웃은 하지 않음"). 같은 장치를 쓰고 싶으시면 별도로 정하겠습니다.

백엔드 160 — `POST /auth/password-reset/request` (재설정 요청 라우터)

항목	내용
파일	`studionx/backend/app/api/v1/auth.py`의 `password_reset_request()`, 스키마 `app/schemas/auth.py`의 `PasswordResetRequest`, `PasswordResetRequestResponse`
작업	P1-28 비밀번호 찾기 (FR-118): 열한 번째 함수
작성	Claude (AI), 2026-10-15, 사용자 검토
이전	159 `/auth/refresh`
시험	`tests/test_password_reset_service.py`의 13개 추가 (파일 전체 56개), 실제 앱과 실제 MySQL
다음	`POST /auth/password-reset/confirm`: 토큰과 새 비밀번호로 `reset_password()`를 부름

1. 하는 일

로그인 없이 부르는 API입니다. 이메일 하나를 받아 `request_password_reset()`(154번)을 부르고, **가입 여부와 관계없이 항상 같은 응답**을 돌려줍니다.

```
POST /api/v1/auth/password-reset/request
{ "email": "you@example.com" }

→ 202 Accepted
{ "message": "If an account exists for that email, we've sent a message." }
```

결과	언제
202 + 고정 본문	형식이 맞는 모든 요청 (가입된 주소, 없는 주소, 탈퇴, 정지 계정 모두 같음)
422 `VALIDATION_ERROR`	이메일 형식이 틀림, 본문이 없음
429 `TOO_MANY_REQUESTS` + `details.retry_after`	요청이 잦음 (가입 여부와 무관하게 셈)

오류는 이 둘뿐이고, 둘 다 가입 여부와 상관없는 이유로만 납니다.

2. 코드

```
class PasswordResetRequest(BaseModel):
    email: EmailStr = Field(max_length=255)

class PasswordResetRequestResponse(BaseModel):
    message: str

PASSWORD_RESET_REQUEST_MESSAGE = "If an account exists for that email, we've sent a message."

@router.post("/password-reset/request", response_model=PasswordResetRequestResponse,
             status_code=status.HTTP_202_ACCEPTED,
             responses={422: {"model": ErrorResponse}, 429: {"model": ErrorResponse}})
def password_reset_request(data: PasswordResetRequest, request: Request, db: Db):
```

```
    password_reset_service.request_password_reset(db, data.email, client_ip(request))
    return PasswordResetRequestResponse(message=PASSWORD_RESET_REQUEST_MESSAGE)
```

3. 왜 이렇게

선택	이유
상태 코드 202 (Accepted)	"요청을 받았다"는 뜻입니다. 메일이 실제로 가는지는 가입 여부에 달려 있고 그것을 알리지 않으므로, 200(성공)이나 201(만들었다)보다 정확합니다
고정 문구	`If an account exists…`는 "있다면 보냈다"는 조건문이라 사실을 드러내지 않습니다. 본문과 응답 길이까지 같습니다(시험으로 확인)
이메일 형식만 검사 (`EmailStr`)	형식 오류(422)는 가입 여부와 무관하게 입력만 보고 나는 오류라 새는 정보가 없습니다. 가입 때와 같은 검사입니다
IP는 문의 라우터의 `client_ip()`를 가져다 씀	같은 규칙(프록시 뒤에서만 `X-Forwarded-For`를 믿음)을 복사하지 않고 한 곳에서 관리합니다. 프록시 설정은 `TRUST_PROXY` 하나로 함께 바뀝니다
응답에 이메일과 토큰을 넣지 않음	응답은 로그, 프록시, 브라우저 기록에 남을 수 있습니다

4. 시험 (13개 추가, 파일 전체 56개 통과)

확인	결과
경로가 등록되고 202, 429가 문서에 있음	✓
가입된 계정은 202 + 고정 본문이고 토큰이 저장됨	✓
없는 계정이 가입된 계정과 똑같은 응답(상태, 본문, `content-length`, 쿠키 없음)	✓
탈퇴, 정지 계정도 같은 응답이고 토큰은 만들지 않음	✓ ✓
이메일 대소문자와 공백을 정리한 뒤 찾음	✓
형식이 틀린 이메일 5종은 422	✓ (5개)
본문이 비었거나 없으면 422	✓
가입 여부와 관계없이 4번째 요청이 429(`retry_after` 포함)	✓
같은 IP에서 서로 다른 이메일 11번째가 429(이메일 한도와 별개의 IP 한도)	✓
로그인 없이 호출됨	✓
응답에 토큰과 이메일이 들어 있지 않음	✓

시험을 확인하는 시험. 일부러 틀리게 바꿔 시험이 잡는지 보았습니다.

바꾼 것	결과
상태 코드를 200으로	8개 실패
가입되지 않은 주소에만 다른 본문("No such account.")	1개 실패
IP 대신 `None`을 넘김	**처음에는 실패 없음**, 시험을 보강한 뒤 1개 실패
횟수 제한 제거	2개 실패
가입되지 않은 주소에서 404	4개 실패

시험을 보강한 일. "IP 대신 `None`을 넘기는" 실수를 처음에는 시험이 잡지 못했습니다. 이메일 한도(3번)는 라우터를 거쳐 시험했지만 IP 한도(10번)는 서비스 함수로만 시험했기 때문입니다. 라우터가 IP를 제대로 넘기지 않으면

IP 한도가 조용히 꺼지는 위험한 빈틈이어서, 라우터를 거쳐 11번째 요청을 확인하는 시험을 더했습니다. 보강 뒤에는 같은 변이를 잡습니다.

전체 pytest 801개 통과, `ruff` 통과.

5. 알아 둔 한계: 응답 시간

응답 내용은 같지만 **걸리는 시간**은 다를 수 있습니다. 가입된 계정은 토큰 저장(DB 쓰기)과 로그 출력을 하고, 없는 계정은 조회만 하고 끝나서 몇 밀리초 빠릅니다. 아주 많은 요청을 정밀하게 재면 가입 여부를 추측할 수 있지만, 이 API는 이메일당 10분에 3번, IP당 10번으로 제한되어 통계를 낼 만큼 부를 수 없습니다. 실제 메일(SMTP)을 붙이면 시간 차가 훨씬 커지므로, 그때 **메일 발송을 응답 뒤의 백그라운드 작업**으로 옮겨야 합니다. 배포 전 할 일로 남겨 둡니다.

백엔드 161 — `POST /auth/password-reset/confirm` (새 비밀번호로 바꾸기 라우터)

항목	내용
파일	`studionx/backend/app/api/v1/auth.py`의 `password_reset_confirm()`, 스키마 `app/schemas/auth.py`의 `PasswordResetConfirm`
작업	P1-28 비밀번호 찾기 (FR-118): 열두 번째 함수. **백엔드의 마지막 API**
작성	Claude (AI), 2026-10-15, 사용자 검토
이전	160 `POST /auth/password-reset/request`
시험	`tests/test_password_reset_service.py`의 15개 추가 (파일 전체 71개), 실제 앱과 실제 MySQL
다음	화면: 로그인 모달의 복구 화면(Password, Email 탭)과 `resetPassword.html`

1. 하는 일

로그인 없이 부르는 API입니다. 메일 링크의 토큰과 새 비밀번호를 받아 `reset_password()`(155번)를 부릅니다.

```
POST /api/v1/auth/password-reset/confirm
{ "token": "<메일 링크의 토큰>", "new_password": "Newpass1!" }

→ 204 No Content (본문 없음)
```

결과	언제
204	비밀번호가 바뀜
400 `INVALID_RESET_LINK` "This link is invalid or has expired."	링크가 틀렸거나, 만료됐거나, 이미 썼거나, 계정 상태가 맞지 않음 (**이유를 구분하지 않는 같은 문구**)
422 `VALIDATION_ERROR` + `details.new_password`	새 비밀번호가 규칙(FR-102)에 맞지 않음. **링크는 그대로 쓸 수 있음**
422	본문이 비었거나 필드가 없음, 토큰이 200자를 넘음

2. 코드

```
class PasswordResetConfirm(BaseModel):
    token: str = Field(min_length=1, max_length=200)
    new_password: str = Field(min_length=1, max_length=100)

@router.post("/password-reset/confirm", status_code=status.HTTP_204_NO_CONTENT,
             responses={400: {"model": ErrorResponse}, 422: {"model": ErrorResponse}})
def password_reset_confirm(data: PasswordResetConfirm, db: Db) -> None:
    password_reset_service.reset_password(db, data.token, data.new_password)
```

3. 결정과 이유

결정	이유
성공해도 자동 로그인을 하지 않음 (사용자가 정함)	메일 링크만 가진 사람이 곧바로 로그인되는 일을 막습니다. 재설정한 사람이 새 비밀번호로 직접 로그인합니다. 응답에 쿠키와 토큰이 없음을 시험으로 확인합니다
필드 이름 `new_password` (사용자가 정함)	비밀번호 변경(FR-107)과 같은 이름입니다
토큰은 길이만 제한하고 형식은 보지 않음	형식이 틀린 값과 없는 값을 구분하면 추측을 돕습니다. 어떤 값이든 서비스가 해시로 확인하고 같은 400으로 답합니다. 200자 제한은 터무니없이 긴 값이 해시 계산으로 가는 것을 막는 상한입니다
이 API에는 횟수 제한을 두지 않음	토큰이 256비트 무작위라 추측으로 맞힐 수 없습니다. 요청 API의 제한은 메일 폭탄을 막으려는 것이고, 이 API에서 틀린 토큰은 DB 조회 한 번으로 끝납니다. 필요해지면 서비스 쪽에 추가합니다
204	본문이 필요 없고, 화면은 성공하면 로그인 안내로 넘어가면 됩니다

4. 시험 (15개 추가, 파일 전체 71개 통과)

확인	결과
경로가 등록되고 204, 400, 422가 문서에 있음	✓
비밀번호가 바뀌고 204이며 본문이 없음	✓
자동 로그인을 하지 않음(쿠키 없음, 응답에 토큰 없음, 토큰 없이 `/users/me`는 401)	✓
재설정 뒤 새 비밀번호로 로그인되고 예전 비밀번호는 거부됨	✓
틀린 토큰은 400이고 문구와 본문이 정해진 그대로	✓
토큰은 한 번만 쓸 수 있음(같은 링크를 두 번 쓰면 두 번째는 400)	✓
만료된 토큰이 틀린 토큰과 **같은 응답**	✓
약한 비밀번호는 422이고 링크는 그대로 쓸 수 있음(고쳐서 다시 보내면 204)	✓
잘못된 본문 6종(비었음, 필드 없음, 빈 값, 200자 초과)은 422	✓ (6개)
전체 흐름: 이전 로그인이 유효 → 요청 API → 로그에 찍힌 링크의 토큰 → 재설정 → 이전 로그인이 **401**	✓

시험을 확인하는 시험. 일부러 틀리게 바꿔 시험이 잡는지 보았습니다.

바꾼 것	결과
상태 코드를 200으로	5개 실패
서비스 호출 제거	7개 실패
토큰과 비밀번호를 서로 뒤바꿔 넘김	5개 실패
토큰 길이 제한 제거	2개 실패
토큰 빈 값 허용	1개 실패

전체 pytest 816개 통과, `ruff` 통과.

5. 백엔드가 끝났습니다

비밀번호 찾기의 백엔드는 여기까지입니다. 함수 12개와 모델, 설정이 모두 시험으로 확인되었습니다.

번호	함수	하는 일
150	`hash_reset_token`	토큰 해시

번호	함수	하는 일
151	`generate_reset_token`	토큰 만들기
152	`create_reset_token`	토큰 저장 (이전 토큰 무효화)
153	`check_reset_rate_limit`	요청 횟수 제한
154	`request_password_reset`	요청 처리, 로그로 메일
155	`reset_password`	비밀번호 바꾸기
156	`issued_before_password_change`	이전 로그인 판단
157	`decode_token_with_iat`	토큰의 발급 시각 꺼내기
158	`get_current_user` 연결	액세스 토큰 거부
159	`/auth/refresh` 연결	재발급 거부, 쿠키 삭제
160	`POST …/request`	요청 API
161	`POST …/confirm`	확인 API

배포 전에 남은 일(각 문서에 적어 둔 것의 모음)

1. 실제 메일 발송(SMTP)을 붙이고 `MAIL_BACKEND`를 바꿔야 합니다. 그 전에는 배포에서 재설정이 동작하지 않습니다(안전한 쪽으로 실패).
2. 메일 발송을 응답 뒤의 백그라운드 작업으로 옮겨야 합니다(응답 시간 차로 가입 여부가 새지 않게).
3. `resetPassword.html`은 `#token=`을 읽은 뒤 주소창에서 지우고(`history.replaceState`), `Referrer-Policy: no-referrer`를 둬야 합니다.
4. 개인정보 처리방침(세 판)에 "재설정 메일 발송" 항목을 반영하고, 요청 횟수 제한이 서버 메모리 방식이라 서버를 여러 대로 늘리면 Redis로 옮겨야 합니다.
5. 이메일을 잊은 경우(FR-119)의 안내 메일은 아직 만들지 않았습니다(화면 작업과 함께 정합니다).

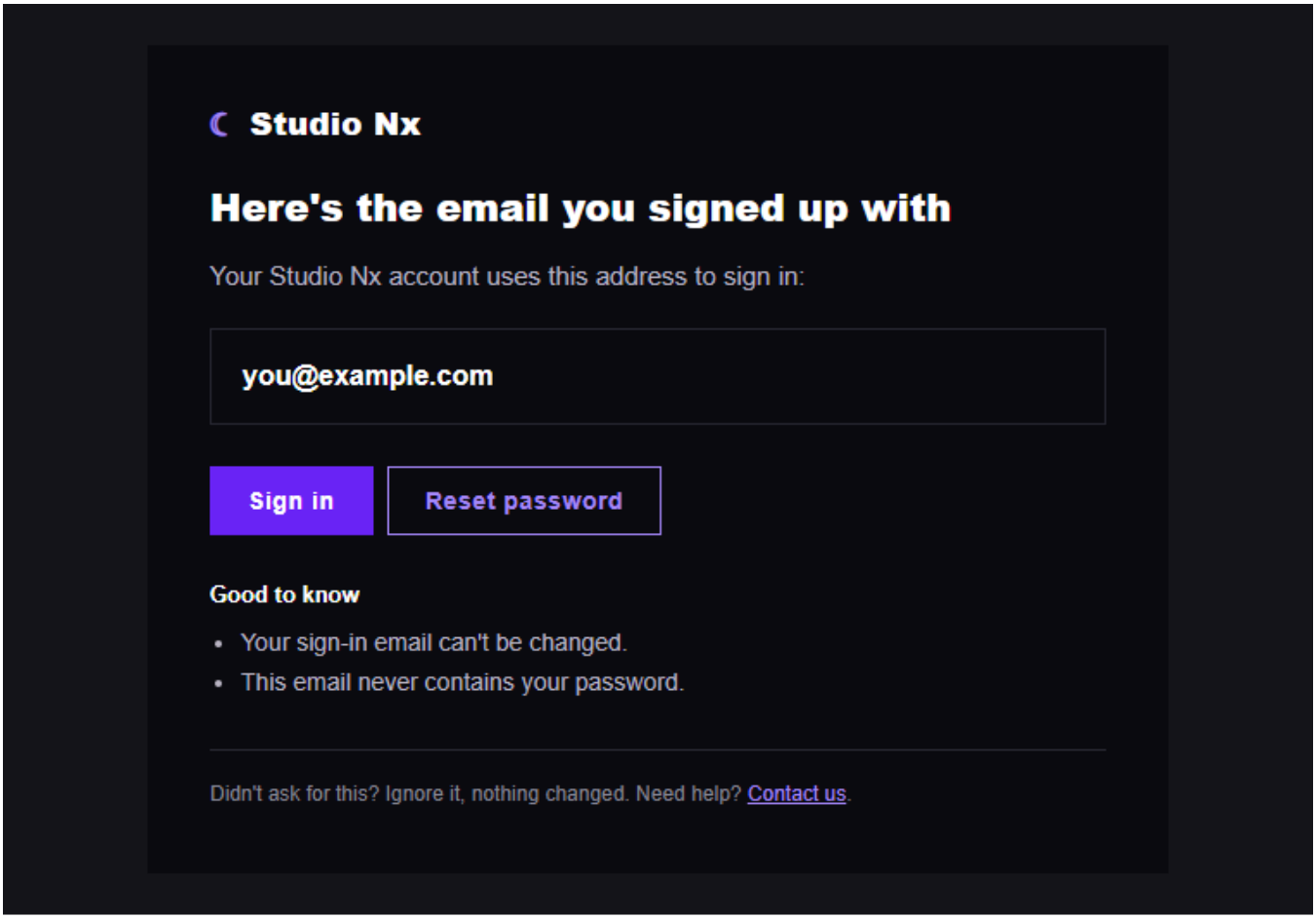
백엔드 162 — `build_email_reminder_message()` (이메일 안내 메일 문안)

항목	내용
파일	`studionx/backend/app/services/mail_templates.py` (새 파일)
작업	P1-28 이메일을 잊은 경우 (FR-119): 열세 번째 함수
작성	Claude (AI), 2026-10-15, 사용자 검토
이전	161 `POST …/confirm`
시험	`tests/test_mail_templates.py` 16개
다음	`request_email_reminder()`: 가입된 `ACTIVE` 계정이면 이 문안으로 메일(로그)을 보내고, 항상 같은 결과를 돌려줌

1. 하는 일

"가입한 이메일이 무엇인지" 알려 주는 메일의 **문안**(제목, 글자판, HTML)을 만듭니다. 보내는 일(전송)과 분리해서 문안만 만들기 때문에, 나중에 SMTP를 붙여도 이 함수는 바뀌지 않습니다.

모양은 사용자가 정했습니다: **C안(도움말을 곁들인 안내)에 B안의 어두운 카드 색을 입힌 것**이고, D안(가입되지 않은 주소에 "계정이 없다"고 알리는 메일)은 만들지 않습니다.



2. 담는 것과 담지 않는 것

담는 것	담지 않는 것
로그인 이메일, `Sign in` 버튼, `Reset password` 버튼	닉네임, 가입일, 프로젝트 이름 (메일함이 남에게 열려 있을 수 있음)
알아 둘 점 2가지("이메일은 바꿀 수 없다", "이 메일에는 비밀번호가 없다")	비밀번호
"요청하지 않았다면 무시하세요"	토큰, `#token=`
`Contact us` 링크(사람이 처리하는 서브 안내, E안)	외부 이미지, 외부 파일

링크는 사이트 주소(`APP_BASE_URL`)의 `home.html#login`(로그인 모달), `home.html#recover`(복구 화면), `home.html#contact`(문의)를 가리킵니다. **`#recover`는 화면 작업에서 만들어야 하는 주소입니다**(복구 모달을 여는 해시, `#login`과 같은 방식).

3. 코드 (요약)

```
@dataclass(frozen=True)
class MailMessage:
    subject: str
    text: str
    html: str

def build_email_reminder_message(email: str, base_url: str) -> MailMessage:
    base = base_url.rstrip("/")
    ...
    shown = escape(email, quote=True)          # HTML에 넣기 전에 이스케이프
    return MailMessage(subject=..., text=..., html=...)
```

4. 왜 이렇게

선택	이유
모든 스타일을 태그에 직접(인라인)	메일 프로그램은 `<style>` 태그를 지우는 일이 많고 CSS 변수도 못 씁니다. 색도 `tokens.css`의 값을 그대로 적었습니다(`#0b0a0f`, `#6a24f6`, `#a78bfa` 등)
글자판(text)도 함께 만듦	HTML을 못 보는 메일 프로그램과 스팸 검사에서 불리하지 않도록 같은 내용의 글자판을 둡니다
이메일을 HTML에 넣을 때 이스케이프	이메일 형식 검사를 통과해도 `a"><script>…@example.com` 같은 값이 마크업으로 해석되는 일을 막습니다. 속성(링크 주소)에도 같은 처리를 합니다
이미지를 쓰지 않음	로고는 글자와 기호(©)로 대신했습니다. 외부 이미지는 많은 메일 프로그램이 기본으로 막고, 추적 픽셀로 오해받을 수 있습니다
문안만 만드는 순수 함수	시험이 빠르고, 전송 방식(로그, SMTP)이 바뀌어도 영향이 없습니다
다른 정보를 넣지 않음	메일함은 본인 것이 아닐 수도 있고(남이 입력한 주소), 이 메일은 "이 주소가 로그인 ID"라는 사실만 알려야 합니다

5. 시험 (16개 통과)

확인	결과
제목, 글자판, HTML을 가진 `MailMessage`를 돌려줌	✓
로그인 이메일이 글자판과 HTML에 모두 있음	✓
링크 세 개(`#login`, `#recover`, `#contact`)가 HTML과 글자판에 있음	✓
기준 주소 끝의 `/`가 겹치지 않음	✓
토큰, 닉네임, 가입일, 프로젝트가 없음	✓
이메일이 HTML에서 이스케이프됨(`<script>`가 마크업이 되지 않음), 글자판은 그대로	✓
`&`도 이스케이프됨	✓
링크 주소가 속성 밖으로 빠져나가지 못함	✓
사이트의 어두운 카드 색을 씀	✓
스타일이 모두 인라인이고 `<style>`, `<link>`, `<script>`, `<img>`, 외부 주소가 없음	✓
HTML이 올바르게 닫혀 있음	✓
목업에서 고른 구성 요소(`Sign in`, `Reset password`, `Contact us`, `Good to know`)가 있음	✓ (4개)
안심 문구와 알아 둘 점이 있음	✓

시험을 확인하는 시험. 일부러 틀리게 바꿔 시험이 잡는지 보았습니다.

바꾼 것	결과
이메일 이스케이프 제거	2개 실패
링크 주소 이스케이프 제거	1개 실패
기준 주소 끝의 `/` 처리 제거	1개 실패
어두운 배경을 흰색으로	1개 실패
`#recover` 링크 제거	1개 실패
본문에 "nickname"을 넣음	1개 실패

전체 pytest 832개 통과, `ruff` 통과.

6. 확인하지 못한 것

실제 메일 프로그램(Gmail, Naver 메일, Outlook)에서의 모습입니다. 헤드리스 Chrome으로 HTML을 그려 본 것(위 그림)만 확인했습니다. 특히 Outlook은 일부 CSS를 무시하므로, 실제 SMTP를 붙일 때 테스트 메일로 확인해야 합니다. 어두운 배경은 메일 프로그램의 다크 모드와 겹치면 다르게 보일 수 있습니다.

백엔드 163 — `request_email_reminder()` (이메일 안내 요청)

항목	내용
파일	`studionx/backend/app/services/password_reset_service.py`
작업	P1-28 이메일을 잊은 경우 (FR-119): 열네 번째 함수 (`check_reminder_rate_limit()`과 내부 `_rate_limit()` 포함)
작성	Claude (AI), 2026-10-15, 사용자 검토
이전	162 `build_email_reminder_message()`
시험	`tests/test_password_reset_service.py`의 11개 추가 (파일 전체 82개)
다음	`POST /auth/email-reminder/request` 라우터

1. 하는 일

"가입한 이메일이 무엇인지" 알려 주는 메일을 요청합니다. 재설정 요청(154번)과 같은 방식이고 **가입 여부와 관계없이 항상 같은 결과**를 돌려줍니다.

순서	일
1	`check_reminder_rate_limit()`으로 횟수를 검사합니다(이메일 3번, IP 10번, 10분). 넘으면 429입니다
2	이메일(소문자, 공백 제거)로 회원을 찾습니다. 없거나 `ACTIVE`가 아니면(탈퇴, 정지) **아무것도 보내지 않고** 끝냅니다
3	`ACTIVE`이면 `build_email_reminder_message()`(162번)로 문안을 만들어 메일(지금은 서버 로그)로 보냅니다

사용자 결정대로 **가입되지 않은 주소에는 아무것도 보내지 않습니다**(D안, "계정이 없다" 메일은 만들지 않음).

2. 코드 (요약)

```
def request_email_reminder(db: Session, email: str, ip: str | None) -> None:
    check_reminder_rate_limit(ip, email)

    user = db.scalar(select(User).where(User.email == email.strip().lower()))
    if user is None or user.status != UserStatus.ACTIVE:
        return

    settings = get_settings()
    if settings.mail_backend != "log":
        logger.warning("email reminder requested but mail is not configured (MAIL_BACKEND)")
        return

    message = build_email_reminder_message(user.email, settings.app_base_url)
    logger.info("[DEV MAIL] to %s: %s", mask_email(user.email), message.subject)
```

3. 결정과 이유

결정	이유
한도는 재설정 요청과 같고 기록은 따로 셈	비밀번호 재설정을 여러 번 눌렀다고 이메일 안내가 막히거나 그 반대가 되지 않습니다. 이 결정은 제안한 대로 진행했습니다

결정	이유
속도 제한 본문을 `_rate_limit(store, ip, email)` 한 곳으로 모음	두 요청이 같은 규칙을 쓰므로 복사하지 않고, 기록 저장소(`store`)만 다르게 넘깁니다. 기존 `check_reset_rate_limit()`의 동작과 시험은 그대로입니다
토큰을 만들지 않음	이 메일은 이미 아는 사실(로그인 이메일)을 알려 줄 뿐이고 아무 권한도 주지 않습니다. 비밀번호를 바꾸려면 메일의 `Reset password` 버튼으로 재설정 요청을 따로 합니다
로그에는 받는 사람을 **가려서** 남기고 **본문은 남기지 않음**	본문에 이메일이 들어 있고, 로그는 오래 남고 여러 곳으로 흘러갑니다. 제목만 남깁니다
`mail_backend`가 `log`가 아니면 아무것도 보내지 않음	재설정 요청과 같은 규칙입니다. 배포에서 메일 설정 없이 켜져 있어도 안전한 쪽으로 실패합니다

4. 시험 (11개 추가, 파일 전체 82개 통과)

확인	결과
`ACTIVE` 회원은 보낸 것으로 기록되고 받는 사람이 가려짐, 로그에 원문 이메일이 없음	✓
로그에 본문(링크)과 토큰이 없고 토큰 행도 만들지 않음	✓
없는 이메일은 아무것도 보내지 않고 같은 결과	✓
탈퇴, 정지 회원은 아무것도 받지 않음	✓ ✓
이메일의 대소문자와 공백을 정리한 뒤 찾음	✓
가입 여부와 무관하게 4번째가 429	✓
재설정 요청과 이메일 안내는 한도를 따로 셈(안내를 다 써도 재설정은 3번 가능)	✓
IP 한도 10번	✓
`mail_backend`가 `log`가 아니면 보내지 않고 "메일 설정 없음"만 기록	✓
문안은 템플릿에서 오고 저장된 이메일과 설정의 사이트 주소를 씀	✓

시험을 확인하는 시험. 일부러 틀리게 바꿔 시험이 잡는지 보았습니다.

바꾼 것	결과
한도 기록을 재설정과 공유	1개 실패
상태 검사 제거	2개 실패
가입되지 않은 주소에도 발송	3개 실패
이메일 가리기 제거	1개 실패
본문을 로그에 남김	2개 실패
`mail_backend` 검사 제거	1개 실패
횟수 검사 제거	3개 실패
토큰을 만들어 버림	1개 실패

시험을 쓰다 발견한 것. 같은 시험 파일에 이름이 같은 함수(`test_email_lookup_ignores_case_and_spaces`)가 둘 있어서 `ruff`가 "다시 정의됨(F811)"을 알렸습니다. 파이썬에서는 뒤의 것이 앞의 것을 덮어 **앞 시험이 조용히 실행되지 않았습니다.** 이름을 `test_reminder_email_lookup_…`으로 바꿔서 둘 다 실행되게 했습니다(시험 수가 81개에서 82개로 늘어난 이유).

전체 pytest 843개 통과, `ruff` 통과.

백엔드 164 — `POST /auth/email-reminder/request` (이메일 안내 요청 라우터)

항목	내용
파일	`studionx/backend/app/api/v1/auth.py`의 `email_reminder_request()`, 스키마 `app/schemas/auth.py`의 `EmailReminderRequest`, `EmailReminderResponse`
작업	P1-28 이메일을 잊은 경우 (FR-119): 열다섯 번째 함수. **백엔드의 마지막 API**
작성	Claude (AI), 2026-10-15, 사용자 검토
이전	163 `request_email_reminder()`
시험	`tests/test_password_reset_service.py`의 15개 추가 (파일 전체 97개), 실제 앱과 실제 MySQL
다음	화면: 로그인 모달의 복구 화면(Password, Email 탭)과 `resetPassword.html`

1. 하는 일

로그인 없이 부르는 API입니다. 이메일 하나를 받아 `request_email_reminder()`(163번)를 부르고 **가입 여부와 관계없이 항상 같은 응답**을 돌려줍니다. 재설정 요청 API(160번)와 같은 모양입니다.

```
POST /api/v1/auth/email-reminder/request
{ "email": "maybe-this-one@example.com" }

→ 202 Accepted
{ "message": "If an account exists for that email, we've sent a message." }
```

결과	언제
202 + 고정 본문	형식이 맞는 모든 요청 (가입된 주소, 없는 주소, 탈퇴, 정지 계정 모두 같음)
422 `VALIDATION_ERROR`	이메일 형식이 틀림, 본문이 없음
429 `TOO_MANY_REQUESTS` + `details.retry_after`	요청이 잦음 (재설정 요청과 **한도를 따로 셈**)

2. 코드

```
class EmailReminderRequest(PasswordResetRequest): ...      # 본문 { email }, 검사는 재설정 요청과 같음
class EmailReminderResponse(PasswordResetRequestResponse): ...

@router.post("/email-reminder/request", response_model=EmailReminderResponse,
             status_code=status.HTTP_202_ACCEPTED,
             responses={422: {"model": ErrorResponse}, 429: {"model": ErrorResponse}})
def email_reminder_request(data: EmailReminderRequest, request: Request, db: Db):
    password_reset_service.request_email_reminder(db, data.email, client_ip(request))
    return EmailReminderResponse(message=PASSWORD_RESET_REQUEST_MESSAGE)
```

3. 왜 이렇게

선택	이유
요청, 응답 모양을 재설정 요청과 **같게** 함	화면(JS)이 두 탭에서 같은 코드로 처리할 수 있습니다. 응답 문구도 같아서 "메일을 보냈다"는 안내를 공유합니다
스키마를 **하위 클래스**로 둠	모양은 같지만 OpenAPI 문서에서 이름이 구분되고, 나중에 한쪽만 바꿔도 다른 쪽이 영향을 받지 않습니다
상태 코드 202, 고정 문구	160번과 같은 이유입니다. 메일이 실제로 가는지는 가입 여부에 달려 있고 그것을 알리지 않습니다
IP는 문의 라우터의 `client_ip()`를 씀	프록시 규칙(`TRUST_PROXY`)을 한 곳에서 관리합니다
응답에 이메일을 되돌려 주지 않음	응답은 로그, 프록시, 브라우저 기록에 남을 수 있습니다

4. 시험 (15개 추가, 파일 전체 97개 통과)

확인	결과
경로가 등록되고 202, 429가 문서에 있음	✓
가입된 계정은 202 + 고정 본문이고 로그에 가려진 받는 사람이 남음	✓
없는 계정이 가입된 계정과 똑같은 응답(상태, 본문, `content-length`, 쿠키 없음), 보낸 것은 가입된 주소 하나뿐	✓
탈퇴, 정지 계정도 같은 응답이고 아무것도 보내지 않음	✓ ✓
형식이 틀린 이메일 5종은 422	✓ (5개)
본문이 없거나 비면 422	✓
가입 여부와 관계없이 4번째 요청이 429	✓
같은 IP에서 서로 다른 이메일 11번째가 429(라우터를 거쳐서)	✓
이메일 안내를 다 써도 재설정 요청은 막히지 않음(라우터를 거쳐서, 한도 분리)	✓
로그인 없이 호출되고 응답에 이메일이 없음	✓

시험을 확인하는 시험. 일부러 틀리게 바꿔 시험이 잡는지 보았습니다.

바꾼 것	결과
상태 코드를 200으로	9개 실패
IP 대신 `None`을 넘김	1개 실패 (160번에서 보강한 IP 시험이 이번에는 처음부터 포함됨)
가입되지 않은 주소에만 다른 본문	1개 실패
서비스를 재설정으로 잘못 연결	2개 실패
한도 기록을 재설정과 공유	2개 실패
가입되지 않은 주소에 404	6개 실패

전체 pytest 858개 통과, `ruff` 통과.

5. 백엔드가 모두 끝났습니다

비밀번호 찾기와 이메일을 잊은 경우의 백엔드가 끝났습니다. 함수 15개, 모델, 설정이 모두 시험으로 확인되었습니다.

번호	내용
150~159	토큰 해시, 생성, 저장, 요청 횟수 제한, 요청 처리, 비밀번호 바꾸기, 이전 로그인 무효화(액세스, 리프레시)
160, 161	`POST /auth/password-reset/request`, `/confirm`
162, 163	이메일 안내 메일 문안, 이메일 안내 요청
164	`POST /auth/email-reminder/request`

화면에서 만들어야 하는 것

- 로그인 모달의 `Can't sign in?`이 복구 화면(Password, Email 탭)을 엽니다.
- `home.html#recover`를 열면 복구 화면이 열려야 합니다. 이메일 안내 메일의 `Reset password` 버튼이 이 주소를 가리킵니다(`#login`과 같은 방식).
- `resetPassword.html`이 `#token=`을 읽고 주소창에서 지웁니다.